Transformer 推理部署横纵分析报告

> **研究对象**：Transformer 推理与部署 **类型**：AI 工程 / 基础设施 **方法论**：横纵分析法（Horizontal-Vertical Analysis） **撰写时间**：2026-04-30

目录

1. 一句话定义
2. 前置知识速览
3. 纵向分析：推理引擎的六年演化 – 前 vLLM 时代（2018-2022）→ vLLM 革命（2023）→ 框架混战（2023-2024）→ 消费级推理爆发（2024-2025）→ 端侧与推理即服务（2025-2026）
4. 横向分析：2026 年的推理部署格局 – 四大引擎生态分化 · 并行策略成熟图谱 · 量化竞赛 · 硬件生态分化 · 推理成本下滑曲线
5. 横纵交汇洞察 – 推理引擎的演化规律 · 推理的摩尔定律 · 未来：推理即计算
6. 实战：面试题、场景题、应用题 – 5 道面试题 · 4 道场景题 · 1 道应用题
7. 信息来源

一、一句话定义

Transformer 推理部署，就是把训练好的模型变成可用的服务——给定输入文本，让模型以自回归方式逐 token 生成输出，并在延迟、吞吐、成本三者之间找到可接受的平衡点。

训练只做一次，推理却要重复无数次。训练看重的是吞吐（GPU 利用率推满即可），推理看重的却是首 token 延迟（TTFT）、生成速率（tokens/s）、并发能力、显存上限。一个 70B 模型训练一次花几百万美元，但上线后每天推理成本可能超过训练摊销。这就是推理部署独立成为一门工程学科的根本原因。

核心矛盾：模型越来越大（405B、671B），但推理设备的内存带宽和容量没跟上。所有推理优化——量化、KV cache 压缩、并行策略、投机解码——本质上都是在跟这个矛盾周旋。

二、前置知识速览

> 完整的 Transformer 架构前置知识请参阅 Transformer 前置知识。以下是与推理部署直接相关的核心概念速查。

KV Cache：自回归推理时，每生成一个 token 都要重新计算所有之前 token 的 Key 和 Value。这显然浪费——所以直接把每个 attention 层的 K、V 矩阵存下来，新 token 只需计算自己的 K、V，拼接上去即可。KV cache 的大小 = 2 × n_layers × n_kv_heads × d_head × seq_len × precision_bytes。70B 模型 4K 上下文单请求就吃掉约 10GB 显存，这是推理部署的第一瓶颈。

自回归生成：Transformer 推理是逐 token 生成的——输出 "hello" 后，把 "hello" 拼回输入再预测下一个 token。这意味着无法完全利用 GPU 的并行计算能力，因为第二步必须等第一步完成。

Continuous Batching：传统 static batching 要等一个 batch 全部生成完才释放资源。Continuous batching 让生成完的请求立刻退出，新请求立刻进入——GPU 始终满载。这是 vLLM 的核心贡献之一，吞吐提升 2–4x。

量化（Quantization）：把模型参数从 FP16（16 bit）压缩到 INT4/INT8/FP8，以精度换显存和带宽。70B 模型 FP16 占 ~140GB，INT4 占 ~35GB——从装不进一张 H100 到一张 H100 搞定。

投机解码（Speculative Decoding）：用小模型（draft model）快速生成候选 token 序列，大模型（target model）一次验证整段。理想情况下 2–3x 加速，输出质量无损。

这些概念是理解后续所有分析的基础。推理部署的演进史，就是围绕这些瓶颈的突围史。

三、纵向分析：推理引擎的六年演化

3.1 前 vLLM 时代（2018–2022）：模型不大，问题不大

2018 年的推理很简单。BERT 340M 参数，一张 V100 32GB 就能跑推理，batch size 开 32 也不在话下。GPT-2 1.5B 参数开始有点压力，但单卡也能跑。那时候推理优化的核心关切是 latency——模型小，显存不是问题。

2019 年，NVIDIA 推出了 **FasterTransformer**。这是业界第一个专为 Transformer 推理优化的 CUDA 库。它用手写 CUDA kernel 替代了 PyTorch 的框架层实现——把 Transformer 的 encoder/decoder 拆成精心调优的 kernel，每一层的数据搬运量降到最低。BERT 推理加速 2–4x，GPT-2 加速 3x。FasterTransformer 后来演化为 TensorRT 的一部分，但核心思路一直没变：**框架通用性 = 性能浪费，手写 kernel = 极致性能**。

FasterTransformer 的问题是它太底层了。你要自己管理 KV cache 的分配和释放，自己做 static batching（batch 大小固定，请求同时进出），遇到变长输入要手动 padding。每个公司最终都自己造了一套推理框架——Meta 有 Triton Inference Server 的定制版，Google 有内部推理栈，阿里有 BladeDISC。

2021–2022 年间，模型规模快速膨胀。GPT-3 175B 要 8 张 A100 才能装下参数，GPT-3 单次推理的 KV cache 就需要几十 GB。推理从 "latency 问题" 变成了 "memory 问题"。FasterTransformer 虽然性能好，但不能解决根

本矛盾——显存不够，管理效率极低。

这个时期的关键认知是：**GPU 显存利用率极低**。static batching 下，每个请求的 KV cache 预先分配最大长度，但实际生成的 token 数量远小于最大长度——碎片率 40–60%。没人想过去改这个，因为大家默认这就是推理的"正常状态"。

3.2 vLLM 的革命（2023）：PagedAttention 打破了什么

2023 年 6 月，UC Berkeley Sky Computing Lab 的 Kwon Woosuk 发布了 vLLM。一个硕士生的毕业设计，成了推理框架的分水岭。

vLLM 的核心洞察来自一篇论文：**"Efficient Memory Management for Large Language Model Serving with PagedAttention"**。

问题很简单：GPU 显存里 KV cache 的碎片率 ~60%。每个请求预分配最大长度的空间（假设 2048 tokens），但大部分请求只生成几十个 token。这就好比酒店给每个客人预留了一整层楼，但客人只住一个房间。

解决方案借鉴了操作系统虚拟内存的页式管理：把 KV cache 切成固定大小的块（page，默认 16 个 token），按需分配。请求的 K、V 在物理显存中不需要连续——逻辑上连续就行，通过 block table 做地址映射。

效果：

- 显存利用率从 ~40% 提升到 ~95%
- 同一个 GPU 可以服务的请求数增加 2–4x
- 更大 batch → 更高吞吐（GPU 计算单元更饱和）

PagedAttention 不是算法创新，是系统工程创新。它没有改动 Transformer 的计算图，也没有发明新的 kernel——只是重新安排了显存的管理方式。但效果比很多算法创新都好，因为它在最浪费的地方下手。

vLLM 还做了第二件事：**Continuous Batching**。传统 static batching 中，一个 batch 所有请求必须同时开始、同时结束。vLLM 的做法是让每个请求独立迭代——生成完了就退出 batch，新请求马上补入。这跟 PagedAttention 天生搭配：页式管理让动态进出 batch 的成本极低。

vLLM 开源（Apache 2.0）后立刻激活了整个推理生态。为什么？因为 2023 年也是 LLaMA 开源的年份。Meta 放出了 LLaMA 权重，整个开源社区突然有了自建推理服务的需求——但大家都不知道怎么高效部署。vLLM 正好填上了这个缺口。

3.3 框架混战（2023–2024）：NVIDIA vs 开源社区

vLLM 的成功让所有人都意识到"推理框架是独立赛道"。2023 年下半年到 2024 年，至少四个主流框架在争夺开发者：

TensorRT-LLM（NVIDIA，2023 年 10 月）。NVIDIA 的官方回答。FasterTransformer 的继任者，加上 PagedAttention 式的内存管理（虽然不叫这个名字）和 Continuous Batching。卖点是手写 CUDA kernel 性能最

优——FP8/INT4 量化 kernel、FlashAttention kernel 全部原生集成。缺陷也很明显：NVIDIA GPU only，闭源组件多，编译一次要半小时，API 不兼容主流生态。

TGI（Text Generation Inference）（HuggingFace，2023 年夏）。HuggingFace 的推理框架，理念跟 HF 的整个生态一致——"别管内核，拿来就用"。集成 Transformers 库和 PEFT（LoRA 合并推理），一条命令部署。性能不如 vLLM 和 TensorRT-LLM，但胜在零配置。小团队和个人开发者用它最多。

Ollama（2023 年底）。目标是"让所有人都能本地跑大模型"。一句话安装，启动就开 API。后端用 llama.cpp（GGUF），支持 macOS、Linux、Windows。模型从 HuggingFace 或 Ollama 库下载，自动量化、自动管理。Ollama 不谈性能——它的价值在于"消除技术门槛"。到 2025 年，Ollama 的 GitHub Stars 超过任何一个推理框架。

llama.cpp（2023 年 3 月）。一个纯 C/C++ 实现，没有 Python 依赖。创始人 Georgi Gerganov 最开始只是想做 LLaMA 的 CPU 推理原型，但 GGUF 格式和广泛的量化支持让它成为消费级部署的事实标准。llama.cpp 在 MacBook 上的推理速度比任何 Python 框架都快——因为纯 C++ 没有解释器开销，MPS（Metal Performance Shaders）后端直接调用 Apple GPU。

到 2024 年中，格局逐渐清晰：

- **vLLM**：开源生态的王者，社区最活跃（400+ 贡献者），支持模型最多
- **TensorRT-LLM**：NVIDIA 生态内性能最强，但门槛最高
- **TGI**：HuggingFace 生态标配，性能一般但易用
- **Ollama + llama.cpp**：消费级部署的唯一选择

框架格局的深层本质不是技术分化，而是算力阶层分裂。能买得起 NVIDIA H100 的企业用 TensorRT-LLM 或 vLLM；用一张 RTX 4090 跑 13B 模型的人用 Ollama；MacBook 用户用 llama.cpp。同一个模型，在不同的硬件上，跑在不同的框架里——这不是"谁更好"，而是"你买得起什么"。

3.4 消费级推理爆发（2024-2025）：大模型走进个人电脑

2024 年是模型民主化的转折点。

催化剂是 **GGUF 格式**。在此之前，模型格式是炼狱：PyTorch 存成 .bin（多个文件）、HuggingFace 用 transformers 加载时还要下载 config、tokenizer 分开。换框架就要转格式，转格式就要掉精度。GGUF 把一切都塞进一个文件——权重、tokenizer、config、甚至量化参数。下载一个文件，加载一个文件，推理就开始了。

llama.cpp 把 GGUF 的易用性和性能结合到了极致：

- CPU 推理：用 ggml 的量化 kernel，INT4 精度下 7B 模型跑出 20+ tokens/s（M3 Max）
- GPU 加速：Metal 后端调用 Apple GPU 的 unified memory，省去 CPU↔GPU 拷贝
- Hybrid 模式：部分层跑 GPU、部分层跑 CPU，大模型在显存不够时自动混合

结果：7B 模型在 MacBook Air 上流畅运行，13B 在 RTX 4090 上 40+ tokens/s，70B 在双路 3090（NVLink 或纯 PCIe）上也能跑。

2023 年 12 月，Apple 发布了 **MLX**——专为 Apple Silicon 设计的深度学习框架。MLX 的独特优势来自 Apple 的统一内存架构：CPU 和 GPU 共享同一块物理内存，不像 NVIDIA 的架构需要显式拷贝数据。M2 Ultra 最高 192GB unified memory，可以加载 70B 模型的 INT4 版本（~40GB）并且还有充裕的 KV cache 空间。这在 NVIDIA 生态里至少需要两张 A100 才能做到。

消费级推理的爆发带来了一个更深远的影响：**使用模式的分化**。当推理成本降到零（本地运行），用户的行为会完全不同——无限次对话、私有数据处理、实验性 prompt 操作，这些在 API 模式下成本敏感的用法在本地变得可行。

3.5 端侧与推理即服务（2025–2026）

2025 年以来，两个方向并行加速。

端侧推理：手机跑模型从"能做"变成"能用"。Qualcomm AI Hub 在 Snapdragon 8 Gen 4 上做了 INT4 优化的 7B 模型推理，3–5 tokens/s。Apple Neural Engine 跑 3B 模型到 15 tokens/s。三星 Galaxy S26 系列内置端侧 7B 模型处理。虽然这个速度不适合对话式交互，但适合离线翻译、摘要、分类等不需要实时反馈的场景。

端侧推理的核心约束不是算力而是带宽——手机内存带宽 ~50 GB/s，远低于 H100 的 ~3.35 TB/s。小模型（<3B）在这个约束下可以做到实用，大模型不行。这个物理约束意味着在可预见的未来，端侧和云端推理不会是替代关系，而是分层关系——简单任务端侧处理，复杂任务云端兜底。

推理即服务：2024–2025 年间出现了一批专做推理的云服务商。Together AI 和 Fireworks 基于 vLLM 搭了优化栈，提供多种开源模型的 API 服务。Together 的吞吐优化做得比自建 vLLM 还好——他们在 vLLM 上叠加了 speculative decoding、自定义调度策略、多 region 负载均衡。

Groq 的 **LPU（Language Processing Unit）** 是 2025 年最出乎意料的玩家。Groq 在 LLaMA 70B 上做到了 500 tokens/s——比 H100 快 5 倍以上。不靠更先进的制程（Groq 用的是 14nm），靠的是"为推理定制"的架构：SRAM 替代 HBM、确定性执行模型（没有 cache miss）、极低延迟的跨芯片互联。代价是显存极小（每张 LPU 只有 230MB SRAM），所以超大模型需要几百张 LPU 级联。

Groq 的商业证明了一个道理：**当市场足够大，总有玩家愿意做专用芯片**。推理市场在 2024–2025 年达到了一年数十亿美元的规模，支撑得起定制硬件的开发成本。

同期的 Cerebras 用晶圆级芯片（Wafer Scale Engine）做推理——一片晶圆就是一个芯片，不需要跨芯片通信。在 GPT-3 175B 推理上延迟极低（~100ms TTFT），但芯片成本极高、功耗惊人。

推理即服务 vs 自建推理的决策拉扯在 2025–2026 年成为企业部署的核心矛盾。API 模式省钱省力，但数据隐私和供应商锁定风险让很多企业选择自建。这个矛盾没有标准答案——它在"数据敏感度 × 规模 × 预算"的三维空间里有一体化的决策面。

四、横向分析：2026 年的推理部署格局

4.1 四大推理引擎的生态分化

六年下来，四个主流引擎形成了清晰的生态位：

维度	vLLM	TensorRT-LLM	TGI	llama.cpp
开源协议	Apache 2.0	自定义（含闭源组件）	Apache 2.0	MIT
GPU 支持	NVIDIA / AMD / Intel	NVIDIA only	NVIDIA / AMD	NVIDIA/AMD/Apple/CPU
CPU 推理	不支持	不支持	不支持	核心能力
量化支持	AWQ / GPTQ / FP8	FP8 / FP4 / INT8	GPTQ / AWQ	GGUF（2-8 bit）
PagedAttention	原生（发明者）	类似实现	类似实现	层内有限支持
Continuous Batching	原生	原生	支持	不支持
多节点推理	支持（vLLM 0.6+）	支持	有限	有限
LoRA 适配	原生支持（S-LoRA）	有限	支持	可通过 LLM 外挂
模型生态	最广	NVIDIA 生态	HuggingFace 生态	GGUF 生态
适用规模	7B-405B	7B-405B	7B-180B	1B-70B

vLLM 在 2025-2026 年赢下了最多份额。不是因为它技术最强——TensorRT-LLM 的单卡性能更好——而是因为三个决定性的非技术因素：

- **Apache 2.0 协议**：企业可以放心集成，不用担心授权问题
- **模型兼容性最广**：新模型出来通常 vLLM 最先支持，因为社区贡献者最多
- **生态包围**：vLLM 被 Docker Kubernetes 生态天然接纳，而 TensorRT-LLM 的编译流程（tensorrt_llm build 需要半小时）跟现代 CI/CD 格格不入

TensorRT-LLM 的定位变成了 "终极性能加速器"——你在 vLLM 上部署好了，如果还不够快，可以用 TensorRT-LLM 的专属 kernel 优化临界路径。但在日常运维里，vLLM 几乎成了默认选择。

4.2 并行策略的成熟图谱

大模型推理不能只靠单卡的算力和显存。当模型装不下一张 GPU 时，就需要并行策略。2026 年，几种并行策略已经分化出清晰的适用范围：

张量并行（Tensor Parallelism, TP）：把单个 Transformer 层的参数按行或列切开，分布在多张 GPU 上。每张卡算一部分，通过 all-reduce 通信聚合结果。TP 的内部通信量极大（每个 attention 计算都需要同步），所以必须在同一节点内（NVLink 连接，带宽 ~900 GB/s），跨节点不行。

流水线并行（Pipeline Parallelism, PP）：把模型的不同层放在不同 GPU 上，数据像流水线一样流经各卡。通信量小（只需要传输中间激活），但存在 bubble 问题——流水线头和尾的 GPU 总有空转。随着 PP 深度增加，bubble 比例上升，所以 PP 的深度一般不超过 8。

序列并行（Sequence Parallelism, SP）：把超长序列（如 128K context）切分到多个 GPU，每个 GPU 算一段的注意力。需要 Ring Attention 或类似的通信模式。适合长上下文场景。

专家并行（Expert Parallelism, EP）：MoE 模型专用。把 different experts 分配到不同的 GPU 上，token 在哪张 GPU 上就由哪张 GPU 上的 expert 处理。问题是 all-to-all 通信——token 可能需要从 GPU A 跳到 GPU B 去处理，跳来跳去通信开销极大。DeepSeek V3 的 DualPipe 本质上就是让通信和计算重叠，掩盖这个开销。

2026 年的通用选型指南：

模型规模	推荐策略	通信瓶颈
<13B	单卡，无需并行	无
13B-70B	TP=4/8（单机内）	NVLink 带宽
70B-405B（Dense）	TP=8 + PP=2/4	NVLink + 跨节点延迟
MoE（200B+）	TP + PP + EP	All-to-all 通信
超长上下文（128K+）	上述 + SP	跨节点带宽

关键洞察：**并行策略的瓶颈已经从"计算够不够"变成了"通信快不快"**。H100 的计算能力（989 TFLOPS FP8）远超 NVLink 带宽（900 GB/s），很多模型的推理实际上是通信瓶颈。这也是为什么 Groq 的确定性互联架构在推理延迟上有优势——它不需要等通信仲裁。

4.3 量化竞赛：从软件压缩到硬件原生

量化是推理部署过去三年进步最快的领域。

GPTQ（Frantar et al., 2023）是第一个实用的 LLM 后训练量化方法。它用二阶信息（Hessian 矩阵）决定每层参数的量化精度，避免了均匀量化的精度塌陷。4 bit 量化下困惑度损失约 0.5——在当时已经是惊人的成果。

AWQ（Lin et al., 2024）比 GPTQ 更好。核心洞察：权重的激活分布不均匀，少数"突出的通道"（salient channels）对精度影响远大于其他通道。AWQ 为这些通道保留更高的精度（比例缩放保护），其他通道正常量化。AWQ 的量化速度快（GPTQ 需要逐层校准数小时，AWQ 只需几分钟），硬件支持也更友好——不需要二阶信息计算。

GGUF 的量化策略是"所有人都在用"的灵活方案。它支持 2–8 bit 的对称/非对称量化、分块缩放、混合精度——模型的不同层可以用不同精度。attention 层对量化敏感所以保留高位宽，FFN 层的某些部分可以量化到更低 bit。这种灵活性让 llama.cpp 用户在硬件受限时有一个"最后一搏"的方案。

2025–2026 年最重大的变化是 NVFP4——NVIDIA Blackwell 架构（B100/B200）中硬件原生的 FP4 格式。之前的量化都是用软件模拟低精度（INT4/INT8），GPU 的 tensor core 不支持原生计算。FP4 是 GPU 硬件直接支持的 4-bit 浮点格式，不需要任何模拟操作。效果：模型文件缩小到 FP16 的 1/4，推理速度显著提升。

这改变了量化的使用方式。以前量化是"一个单独的优化步骤"——训练完模型，跑 GPTQ/AWQ，验证精度，再部署。NVFP4 把量化变成了"模型加载时自动降级的一个选项"——你用 FP16 训练，部署时指定 FP4 格式，GPU 硬件直接处理。虽然 NVFP4 的应用还处于早期（Blackwell 刚刚量产），但这个趋势是明确的：**量化正在从软件工程变成硬件特性**。

量化发展的整体路线：

时代	代表方案	bit	方法	精度损失
2022	FP16（基线）	16	无量化	0%
2023	GPTQ, SmoothQuant	4–8	软件后训练量化	0.5–1%
2024	AWQ, GGUF	2–8	基于激活分布的混合精度	0.3–2%
2025	NVFP4, MXFP4	4	硬件原生支持	0.5–1%
2026	FP4 正在普及，FP2/Floating Point 2 在研究中	—	—	—

4.4 硬件生态的分化：从 CUDA 大一统到多架构并存

2020 年的推理硬件只有两个选择：NVIDIA GPU 或者 NVIDIA GPU。2026 年硬件生态已经明显分化。

NVIDIA（统治地位）：H100 单卡推理 FP8 算力 989 TFLOPS，H200 把 HBM 从 80GB 升级到 141GB（HBM3e，带宽 4.8 TB/s），等效于一块 GPU 可以承载更多模型（不用跨卡）。Blackwell（B100/B200，2025 年出货）统一 HBM 池让两张 GPU 共享显存，单机推理能力翻倍。NVIDIA 的优势是生态完善：CUDA +

TensorRT-LLM + Triton Inference Server + NVLink，从头到尾一个闭环。缺点是贵——H100 一张 ~$30,000，B200 预计 ~$50,000。

Apple（统一内存优势）：M2 Ultra 的 192GB unified memory 是一个古怪但有价值的产品。古怪在于它用 HBM 的方式（统一内存）连接 CPU 和 GPU，跟 NVIDIA 的显存架构完全不同。价值在于 192GB 的容量可以运行 70B 模型（INT4）并且还有空间给 KV cache。M3/M4 Ultra 进一步提升了内存带宽（M2 Ultra ~800 GB/s，M3 Ultra 初步信息 ~1.2 TB/s）。Apple 在推理市场的地位是"二级部署或本地原型验证的最佳选择"——不需要买两张 A100，一台 Mac Studio 就行。

AMD（挑战者）：MI300X（2024）用 192GB HBM3 和 5.2 TB/s 带宽直接挑战 H100。价格只有 H100 的 ~60%，理论性能差距在 10-20% 以内。问题在软件生态——ROCm 跟 CUDA 的差距在缩小但依然存在。vLLM 和 PyTorch 在 AMD 上的支持在 2025 年大幅改善，但很多 CUDA 专属优化（如 FlashAttention 的某些 kernel）在 AMD 上需要额外适配。AMD 的市场策略是"足够好 × 足够便宜"——对于优先考虑成本的大规模部署，MI300X 是越来越有吸引力的选择。

定制芯片（细分玩家）：Groq LPU（500 tokens/s on LLaMA 70B）、Cerebras WSE-3（4 倍晶圆级芯片，低延迟）、SambaNova SN40L（可重构数据流架构）。这些定制芯片在特定场景下的性能远超 GPU，但生态太小——不支持 PyTorch、不支持主流推理框架、模型适配需要定制工程。定制芯片的商业化路径是"在少数大客户场景中建立信任，然后逐步扩展"——这跟 NVIDIA 的"先提供通用平台再卷性能"的策略完全相反。

NPU（新兴力量）：Qualcomm AI Engine、MediaTek APU、Apple Neural Engine。这些专为端侧推理设计的硬件在能效比上远胜 GPU。Qualcomm Hexagon NPU 在 INT4 下跑 7B 模型只用 5W——H100 的峰值功耗是 700W。端侧 NPU 的演进方向是"小模型（<7B）能够实时推理"——2026 年已经接近实用。

硬件格局的不确定性在增加。CUDA 的护城河足够深，但 NVIDIA 的高昂定价正在为竞争者创造空间。

4.5 推理成本的下滑曲线

过去三年，推理成本的下降速度超过了大部分人的预期。

2023 年：GPT-4 API 价格 ~$30/1M tokens。自建推理部署 70B 模型需要 4x A100，GPU 租赁成本 ~$40/小时。尝试做任何大规模推理应用的创业公司，月账单轻松突破十万美金。

2024 年：GPT-4o 发布，价格降到 ~$10/1M tokens。DeepSeek V3 的 API 价格 ~$0.5/1M tokens——比 GPT-4 便宜 60 倍。DeepSeek V3 证明了两件事：MoE 让推理成本直接跟激活参数挂钩（37B 激活），而且中资云的价格本身就低。Llama 3 405B 的开源让自建部署有了一个大模型选项——虽然 405B 的推理成本不低（8x H100），但至少比 GPT-4 的闭源调用灵活得多。

2025-2026 年：推理 API 价格持续下降，竞争格局比 2023 年密集 10 倍。Together AI、Fireworks、Groq、DeepInfra、Anyscale 都在抢推理 API 市场。开源模型的质量持续提升（Llama 4、Mistral、Qwen3），闭源模型的溢价空间在缩小。推理还出现了一种新结构——**推理时计算**（o1/o3、Claude Extended Thinking、DeepSeek-

R1)，用户的 query 不是一次推理结束，而是一个涉及多次推理和回溯的循环过程。这种模式下单次推理的"成本"不再是全部——关键指标变成每次思考的性价比。

价格数字：

时间	模型	价格（/1M tokens）	同比降幅
2023-03	GPT-4	~$30	基线
2024-05	GPT-4o	~$10	67%
2024-11	DeepSeek V3	~$0.5	98%
2025-06	Llama 4 API	~$0.3	99%
2026-04	开源模型自建	~$0.05-0.1（电费）	99.7%

驱动成本下降的因素有四个：量化进步（FP16→INT4 让模型缩小 4x 但精度几乎无损）、框架优化（从裸 PyTorch 到 vLLM，相同硬件下吞吐提升 5-10x）、硬件竞争（NVIDIA/AMD/定制芯片的价格拉扯）、MoE 效率（等质量下激活参数只有 dense 的 5-10%）。

趋势：推理成本每 12-18 个月降低 50%，类似摩尔定律但节奏更快。因为驱动力来自四个独立维度——算法、框架、硬件、商业竞争——任何单一维度的放缓都被其他维度补偿。

五、横纵交汇洞察

5.1 推理引擎的演化规律

回顾六年的发展，每个重大突破都来自一个共同模式：**找到"大家习以为常的浪费"，然后把浪费消除**。

vLLM 的 PagedAttention 是一个完美案例。在 vLLM 之前，所有推理框架都接受显存 40-60% 的碎片率——因为"反正模型参数已经占了那么多显存，碎片多一点少一点差别不大"。vLLM 团队不认同这个假设，深入一查发现碎片浪费的显存比模型参数本身还多。消除碎片后，同样的硬件可以服务 2-4 倍的请求。

FlashAttention 是另一个案例。注意力计算中 HBM↔SRAM 的搬运浪费了 ~80% 的时间，但大家认为"算注意力就是要搬数据"——因为人人都这么写，人人都这样优化。Tri Dao 重新排了计算顺序，把搬运降到最低。

Continuous Batching 也是——GPU 的空转时间（等一个 batch 里最慢的请求）成了理所当然的"框架开销"，没人想过让 batch 动态进出。

规律：推理引擎的最优优化往往不是发明新算子，而是消除不必要的冗余。这个规律在 2026 年依然成立——未来最有可能的突破方向，可能不是发明新的量化方案或 kernel，而是重新审视"哪些资源分配是默认但多余的"。

5.2 推理的"摩尔定律"

推理成本正在以接近摩尔定律的速度下降，而且驱动力更加多元化。

驱动力	典型进展	效果
算法优化	FlashAttention、PagedAttention	相同硬件吞吐 3–5x
量化	FP16→INT4→FP4	模型体积 4x 缩小，带宽需求 4x 降低
硬件进步	H100→B200，HBM 带宽提升	每年推理性能 ~1.5x
竞争	多云 + 开源模型 + API 价格战	推理 API 价格年降 50%+

这个组合比传统摩尔定律（只靠晶体管密度翻倍）更健康——当某一个驱动力放缓，其他驱动力可以接力。2024–2025 年量化的进步很大，但这个维度的收益正在递减（从 FP16 到 INT8 收益 2x，INT8 到 INT4 再 2x，再往下 FP4 收益可能只有 1.3–1.5x 且精损失控）。但算法优化（如 speculative decoding 的广泛应用）和硬件竞争（AMD 追赶 + 定制芯片入场）还在加速。

预测：2028 年的推理成本将是 2026 年的 1/10。依据来自四个方向都在以独立节奏推进——硬件每代性能提升 30–50%（NVIDIA、AMD）、框架每年优化 20–40%（vLLM、TensorRT-LLM）、量化每 1–2 年推进一步（FP8→FP4→FP2 研究）、商业竞争让价格每年降 50%。这些效果叠加，保守估计每 2 年 10x 的降幅是合理的。

5.3 未来：推理即计算

2025–2026 年出现了一个更根本的变化，正在重新定义推理部署的设计目标。

这个变化是 **reasoning-time compute（推理时计算）** 的崛起。

传统模型是"一次推理出答案"——输入 query，走一遍前向传播，输出 token，结束。o1/o3、Claude Extended Thinking、DeepSeek-R1 打破了这种模式。模型在推理时做了多轮"内部思考"——生成思考链、回溯、验证、修正——然后才输出最终答案。一组数据显示控制变量下 DeepSeek-R1 要把推理成本提升至少 3x，Claude 的 Extended Thinking token 消耗量达到 5–10x。

这改变了推理引擎的设计目标。传统推理引擎追求**最小化延迟**——越快越好。推理时计算需要的是**自适应地分配计算预算**——简单问题快速通过，复杂问题深入思考。这两个目标存在冲突：如果引擎默认把延迟压到最低，就没有给"思考"留出时间。如果给思考留时间，简单问题的响应时间就被拖慢了。

所以下一个推理引擎的核心能力可能不是"多快"，而是"多聪明地分配计算"。这可能意味着：

- 动态批处理策略：简单请求和复杂请求走不同的调度队列
- 可抢占的推理流程：如果发现模型在"空转思考"（生成无意义的 tokens），可以提前终止

- 思考预算管理：根据问题复杂度分配不同的思考上限
- 验证和回滚：在思考过程中发现错误路径时可以回滚到上一个 check point

这不是推测。vLLM 的 roadmap 中已经出现了 reasoning 相关的 feature 计划。推理引擎的下一波竞争可能不是关于 kernel 优化，而是关于**计算策略的智能调度**。

一个更深远的推论：**推理不再只是一个"需要最小化的成本"，而是一个"需要 scale 的能力"**。

预训练 scaling 正在进入边际递减阶段，但推理时计算 scaling 才刚刚开始。假设一个用户通过 Claude 处理一份 100 页的合同，传统推理需要 1 次调用（分析总结），推理时计算可能需要 10–20 次内部审视（逐条分析、交叉验证、矛盾检测）——这 10–20x 的计算量不是浪费，而是产生额外价值的必要成本。

这意味着"推理部署"这个职能在 AI 公司内部的权重会持续上升。它不再是一个支撑团队，而是直接影响产品形态和体验的核心工程团队。2026 年各大 AI 公司的基础设施招聘也印证了这一点——推理工程师的缺口远大于训练工程师。

六、实战：面试题、场景题、应用题

面试题

Q1: Continuous Batching 和 Static Batching 的区别?

考察对推理引擎核心原理的理解，区分"了解 API"和"理解原理"的候选。

Static Batching：一批请求同时进入推理引擎，全部生成完成后一起释放资源。问题在于不同请求的生成长度不同——短的生成了 10 个 token 就要等长的生成 500 个。GPU 长时间处于"等 straggler"的状态，利用率低。最大 batch 受限于 batch 中所有请求的 KV cache 之和。

Continuous Batching（也叫 In-flight Batching 或 Iteration-level Batching）：以迭代为单位而非批次为单位。每一轮迭代，生成完成的请求退出 batch，新请求进入 batch。GPU 每一轮迭代都满载。PagedAttention 使这成为可能——页式管理中释放 page 和分配 page 的成本极低，动态进出 batch 没有显存调整的开销。

实际收益：相同硬件 + 相同模型，continuous batching 比 static batching 吞吐提升 2–4 倍。

一个被忽视的问题：continuous batching 需要更精细的调度策略。如果不公平调度，长请求可能永远被短请求抢占（starvation）。vLLM 的默认策略是 FIFO + 最大等待时间，但这个不是最优解——生产中需要根据业务需求定制。

Q2: 一个 70B 模型在 4 张 A100 80GB 上推理，显存怎么分?

考察对模型并行、KV cache、quantization 的综合理解。典型的推理显存分配计算题。

第一步：确定并行策略。70B 模型 FP16 占 ~140GB，一张 A100 装不下。需要并行。TP=4（四卡并行）或 TP=2+PP=2。假设 TP=4。

第二步：参数显存。70B × 2 bytes/param = 140GB。TP=4 下每张 GPU 存 1/4 参数，即 140/4 = **35GB**。但 TP=4 需要多加一层通信缓冲区（约 2–3GB）。

第三步：KV cache 显存。假设 LLaMA 3 70B（80 层，GQA 8 KV heads，d=128），单请求 4096 context。KV cache per token per layer = 2 × 8 × 128 × 2 = 4KB。80 层 × 4096 tokens × 4KB = 1.3GB per request。TP=4 下每张 GPU 存 1/4 的 KV head，所以每卡 ~0.33GB/request。4 张 A100 80GB 有 320GB，但不要用满（留 10–15% 给计算中间变量和 CUDA context）。假设用 ~240GB，剩下约 36–50GB 给 KV cache。按 0.33GB/request，可以开 ~110–150 个 concurrent requests。注意这是纯连续批处理场景——生成短回答的可能到 200+ concurrent。

第四步：额外显存。CUDA context ~1–2GB，中间激活变量 ~2–4GB，通信缓冲区（TP all-reduce）~2–3GB。总共约 5–10GB。

总结：4 张 A100 80GB 跑 LLaMA 70B（INT8 量化可降到 70GB，一张就够了；INT4 约 35GB，1 张卡就够）。

Q3: PagedAttention 解决了什么问题？为什么之前没人想到？

考察对显存管理痛点的理解和工程直觉。

解决的问题：推理时 KV cache 的显存碎片和低效管理。Static allocation 模式下，每个请求的 KV cache 预分配最大长度（如 2048 tokens × 模型配置），但大部分请求生成 50–200 tokens 就结束了——浪费了 90% 以上的预留空间。加上不同请求长度不同，碎片率 ~60%。

为什么之前没人想到？三个原因：

1. **思维惯性**：GPU 显存管理一直用"连续大块分配"模式。做推理加速的人专注于 kernel 优化、算子融合，没人从操作系统虚拟内存的角度去想这个问题。
2. **模型不够大**：参数只有几个 GB 到几十 GB 时，KV cache 浪费几十 MB 不算什么。当模型到了 100GB+ 级别，KV cache 从几 MB 涨到几十 GB，浪费才变得不可接受。
3. **动机错位**：做推理框架的团队（FasterTransformer、TensorRT）来自 NVIDIA，他们的自然倾向是"让 kernel 跑更快"，而不是"改内存分配策略"。伯克利这个学术团队没有利益惯性。

所以 PagedAttention 是一个经典的"越界思维"案例——从完全不同的领域（OS 内存管理）借来方案，解决了一个 GPU 推理领域长期存在但没人正视的问题。

Q4: 量化到 INT4 后模型变慢的可能原因？

考察对 quantization overhead 的理解，区分"模型变小 = 变快"的一刀切认知。

INT4 量化后模型体积缩小到 FP16 的 1/4，理论上推理速度应该提升。但实践中可能变慢：

第一，没有硬件原生 INT4 支持。在 Blackwell（B100/B200）之前，NVIDIA GPU 的 tensor core 不支持原生 INT4 矩阵乘法。INT4 推理需要先用 dequantize kernel 把数据解压到 FP16，再做 FP16 计算——dequantize 本身有开销。如果模型带宽敏感度高（小 batch），dequantize 开销可能吞噬掉数据体积减小的全部收益。

第二，反量化开销随 batch 放大的非线性增长。量化参数（scale、zero point）是每张卡、每层、每个 weight block 一套，在量化 kernel 里反复调用。当这个调用链变长（TP + PP + EP），量化参数的管理变得惊人复杂。

第三，不兼容 FlashAttention 等优化。部分 INT4 kernel 无法利用 FlashAttention 的 tiling 优化。相当于你省了模型加载的带宽，但丢了计算的优化。两个核心工程维度的对冲。

第四，精度损失导致模型行为异常。量化后的模型可能陷入更长的推理路径来补偿精度损失——生成更多无效 token、反复修正自己。

Checklist：量化后性能倒退，检查：GPU 是否支持原生低精度计算（Ada 和 Hopper FP8，Blackwell FP4）→ batch size 够大吗（小 batch 量化收益不大）→ 是否做了层间混合精度（attention 放高位宽，FFN 放低位宽）→ 量化 calibration 数据集是否覆盖了推理时的分布。

Q5: 在线客服的推理服务，延迟和吞吐怎么权衡?

这是一道综合题，没有标准答案。考察候选能不能在约束下做合理决策。

约束：客服场景需要 <300ms TTFT（用户等太久会不耐烦），稳定输出（不能中途断流），日均 10 万次对话（约每秒 10+ 并发）。

权衡框架：

1. **模型选型优先于优化**：13B 以下 Dense 模型 > 70B MoE。小模型的 TTFT 天然低。MoE 大模型虽然能力更强，但首次路由（决定哪个 expert 来处理）会增加延迟不确定性。客服场景不需要 70B+ 的能力。
2. **Speculative Decoding 是小模型的最佳加速方案**：小模型部署小 draft model（如 0.5B），能以 2–3x 加速推理且输出完全一致。相比量化（精度不可逆损失），安全的加速方案在延迟敏感场景是第一选择。
3. **Prefix Cache**：客服场景 system prompt 固定，Automatic Prefix Caching（vLLM 内置）可以省掉每次重复计算 system prompt 的 KV cache。TTFT 直接从 150ms 降到 50ms。
4. **Cache 策略**：高频问题（如"你好"、"如何退货"）的响应直接走 cache，不走模型推理。这在客服场景中能过滤 20–30% 的请求。
5. **量化不是首选**：在延迟场景下，模型需要的是快速响应——量化带来的显存收益在低并发时意义不大。如果有余量，FP8 优于 INT4（FP8 有硬件原生支持且精度损失小）。

不推荐的方案：vLLM + Continuous Batching 的大吞吐策略在这个场景有问题——Continuous Batching 最大化的是吞吐，不是单个请求的延迟。当 batch 堆起来，长请求会拖慢短请求。在线客服需要的是延迟稳定，不是最大吞吐。可以考虑 Fixed-size batching + 高优先级通道，或者把请求排队时间和推理调用分离（异步）。

场景题

场景 1：选推理框架

> 公司要做 70B 模型的 API 服务，日均百万请求，团队熟悉 PyTorch。选什么框架？怎么部署？

框架选择：vLLM。理由： 1. PyTorch 生态亲和：vLLM 本身用 PyTorch 实现，团队熟悉 PyTorch 就可以做定制开发和调优 2. Apache 2.0 协议：没有商业授权顾虑 3. 模型兼容性最广：70B 的常见版本（LLaMA 3、Mistral Large、Qwen3）都能直接跑 4. 社区活跃：遇到问题能找到答案

TensorRT-LLM 的部署会更慢（编译流程复杂），TGI 的性能不够，Ollama 不适合 API 服务。

部署方案：4 张 H100（80GB）单机部署。70B INT4 约 35-40GB，8 张卡不并行也行（INT4 单卡能放下），但为了扛住并发需要：TP=4（张量并行减少每卡 KV cache 压力）+ 量化到 INT4。再用一个前缀 cache（system prompt 复用）进一步压延迟。

吞吐估算：INT4 + TP4 + Continuous Batching，单卡 ~300 tokens/s（H100 上 LLaMA 70B），4 卡约 1000+ tokens/s。日均百万请求，每个请求平均 300 tokens 输入 + 200 tokens 输出 = 500 tokens。日均 500M tokens。需要 ~500M / 1000 = 500,000s，约 6 天。但 Continuous Batching 和并发设计可以把日处理量压缩到 ~5-8 小时。再加一组 hot standby 做 failover 和负载均衡。

场景 2：估算显存

> LLaMA 3 70B（80 层，GQA 8 KV head，d=128），4K context，continuous batching 32 请求。需要多少张 A100 80GB?

逐项计算（FP16）：

模型参数：70B × 2 bytes = 140GB。如果 TP=8（8 卡并行），每卡 140/8 = 17.5GB。若 TP=4（4 卡并行），每卡 35GB。

KV cache per request：80 层 × 2（K、V）× 8 KV heads × 128 d_head × 4096 context × 2 bytes = 80 × 2 × 8 × 128 × 4096 × 2 = 1.34GB。

32 请求：1.34 × 32 = 42.9GB。TP 分掉后每卡：42.9/8（若 TP=8）= 5.4GB，或 42.9/4（若 TP=4）= 10.7GB。

每卡总显存（TP=8）：参数 17.5GB + KV cache 5.4GB + 中间变量 & 通信 buffer ~5GB = **~28GB**。A100 80GB 绰绰有余。

每卡总显存（TP=4）：参数 35GB + KV cache 10.7GB + 中间变量 ~5GB = **~51GB**。也在 A100 80GB 范围内，但余量不大。

结论：TP=8 用 8 张 A100 最稳妥，TP=4 用 4 张 A100 也能跑但余量少。如果考虑更长的 context（8K）或更大的 batch，建议 TP=8 方案。

如果你说"INT4 量化后一张卡就够"——那这题过了。

场景 3：设计推理服务

> 在线客服推理服务，要求 <200ms TTFT，支持流式输出，日均 10 万对话。给完整方案。

模型选型：13B Dense Transformer，INT8 或 FP8。这个选择的理由是：客服场景不需要 70B+ 级别能力（主要处理退货、发货、产品咨询），小模型 TTFT 天然低于大模型。13B FP8 约 13GB，单卡 A10（24GB）或 L40S 即可。

量化策略：FP8（硬件原生支持，精度损失 <0.5%）。不加 INT4（在延迟场景中反量化开销不值得）。KV cache 量化到 INT8（压缩一半但对生成质量几乎没有影响）。

并行策略：不需要并行。13B FP8 单卡 13GB，加上 KV cache 和中间变量，A10 24GB 够用。

框架选择：vLLM + Prefix Caching。客服 system prompt 固定，Prefix caching 把 TTFT 从 ~200ms 降到 ~50ms。

缓存策略：用 Redis 缓存高频 query 的完整回复（命中率 20–30%）——"退款多少天到账"这类问题不需要每次都调模型。

架构：API Gateway → Redis Cache（高频问题命中）→ vLLM + 13B 模型 → 流式输出 → 客户端。

部署拓扑：2 台 A10 实例（一台 active，一台 hot standby），Nginx 负载均衡。

SLA 保障：TTFT P99 <200ms，TPS >50（日均 10 万 = 约每秒 1–2，但还要考虑高峰），流式稳定输出。

场景 4：混合部署

> 日常用 7B 模型处理简单请求，检测到复杂问题时自动路由到 70B 模型。

这道题测试对"分层推理"架构的理解，以及兜底设计。

架构：

```
请求 → Router (Classify) → [简单] → 7B fast (vLLM + INT4)
                          → [复杂] → 70B heavy (vLLM + INT4 + TP=4)
```

Router 实现：用一个小分类模型（如 BERT 或 3B LLM）判断请求复杂度。或者简单规则——含专有名词、关联上下文超过 3 轮、query 长度 > 100 chars → 走 70B。Router 本身的延迟应 <50ms。

7B 模型：INT4 量化（~4GB），单卡 A10，TTFT <100ms。负责简单问答。

70B 模型：量化到 INT4（~40GB），TP=4 部署。可以跑在单机 4 卡 A100 上。关键：可以用 Shared KV cache 实例来动态调度 2 个并发请求。

兜底策略：

1. **超时兜底**：70B 推理超过 5s → 降级到 7B 模型，返回快速但可能不够准确的答复
2. **路由误判兜底**：如果 7B 模型对自己的输出置信度过低（log probability < threshold），自动触发一次 70B 的二次验证
3. **流量高峰兜底**：70B 队列堆积 > 5 个请求 → 新来的复杂请求也走 7B（延迟优先于准确率）
4. **服务降级**：两个模型都不可用时 → 走 rule-based 回复（预设 FAQ 模板），确保业务不中断

效果：80% 的简单请求走 7B（成本低、响应快），20% 的复杂请求走 70B。整体推理成本比全部用 70B 降低约 4-5 倍，比全部用 7B 提升复杂场景的准确率约 15-20%（实际数据因业务而异）。

应用题

应用 1：设计一个 KV Cache 调度策略

> 你的推理服务有 100 个并发用户，服务 70B 模型，每张 GPU 只能同时缓存 20 个用户的 KV cache。设计调度策略。

问题本质：KV cache 是有限资源。100 个用户的 KV cache 不足以全部缓存（假设系统只有 2 张 GPU × 内存上限显著小于 100 × 单用户 KV cache），但用户可能不是同时活跃。

分层方案：

1. **活跃/非活跃区分**：最近 30 秒内没有 token 生成的用户标记为非活跃。非活跃用户的 KV cache 可以被驱逐（evict）到 CPU 内存或持久化到磁盘。驱逐后如果用户重新激活，需要从 0 开始重建 KV cache（重建成本 = 重新计算之前所有 token 的 K、V，约 100-150ms）。

2. **LRU + 优先级**：在 GPU 内存中维护一个 LRU（最近最少使用）队列。活跃用户按最近活跃时间排序。当新的活跃用户进入时，驱逐 LRU 队尾的非活跃用户。但给长对话用户（已经生成了 1000+ token）更高的保留优先级——重建成本跟已生成 token 数量成正比。

3. **选择性驱逐**：被驱逐的 KV cache 不直接丢弃，压缩后存到 CPU RAM（INT4 量化 + 异步压缩，单用户从 ~1.3GB 降到 ~350MB）。CPU RAM 远大于 GPU 显存（64GB vs 80GB），可以存更多用户的压缩 KV cache。

4. **预热重建**：用户重新激活时，不用等到第一个 token 生成才开始重建——通过用户的"回到对话"操作可以预判，提前把压缩 KV cache 从 CPU 解压到 GPU。

效果：2 张 GPU 显存（160GB）理论上只能存 ~120 个用户的 FP16 KV cache（1.3GB/用户）。通过 CPU 缓存 + LRU 驱逐，实际可供 500+ 活跃用户的 KV cache 存储。

现实警告：KV cache 交换方案（GPU↔CPU）的 throughput 瓶颈在 PCIe 带宽（~64 GB/s）。当大量用户在短时间内切换活跃状态，PCIe 会成为新的瓶颈。所以这个方案的本质是在"KV cache 显存不够"和"PCIe 带宽有限"之间找平衡——而不是消除瓶颈。

七、信息来源

核心论文

- Kwon et al., "Efficient Memory Management for Large Language Model Serving with PagedAttention" (2023) — https://arxiv.org/abs/2309.06180
- Dao et al., "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness" (2022) — https://arxiv.org/abs/2205.14135
- Frantar et al., "GPTQ: Accurate Post-Training Quantization for Generative Pre-Trained Transformers" (2023) — https://arxiv.org/abs/2210.17323
- Lin et al., "AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration" (2024) — https://arxiv.org/abs/2306.00978
- Leviathan et al., "Fast Inference from Transformers via Speculative Decoding" (2023) — https://arxiv.org/abs/2211.17192
- Pope et al., "Efficiently Scaling Transformer Inference" (2023) — https://arxiv.org/abs/2211.05102
- Sheng et al., "S-LoRA: Serving Thousands of Concurrent LoRA Adapters" (2023) — https://arxiv.org/abs/2311.03285
- Zhong et al., "DISTFLASHATTN: Distributed Memory-efficient Attention for Long-context LLMs Training" (2024) — https://arxiv.org/abs/2310.03294

推理框架

- vLLM 官方文档 — https://docs.vllm.ai/
- TensorRT-LLM 官方文档 — https://nvidia.github.io/TensorRT-LLM/
- TGI 官方文档 — https://huggingface.co/docs/text-generation-inference

- llama.cpp 仓库 — https://github.com/ggerganov/llama.cpp
- Ollama 官方文档 — https://ollama.ai/
- MLX 官方文档 — https://ml-explore.github.io/mlx/

硬件架构

- NVIDIA H100 Tensor Core GPU 架构白皮书 — https://resources.nvidia.com/en-us-tensor-core
- NVIDIA Blackwell GPU 架构 — https://www.nvidia.com/en-us/data-center/technologies/blackwell-architecture/
- Apple M3 Ultra 技术指标 — https://www.apple.com/mac-studio/
- AMD MI300X Instinct 加速器 — https://www.amd.com/en/products/accelerators/instinct/mi300x.html
- Groq LPU 推理架构 — https://groq.com/

分析报道

- "The Infrastructure Behind AI: A Comprehensive Guide to LLM Inference" — SemiAnalysis (2024)
- "LLM Inference Performance Engineering: From Framework to Hardware" — Anyscale (2024) — https://www.anyscale.com/blog/llm-inference-performance-engineering
- "vLLM: A Year of Serving the Open-Source Community" — Berkeley Sky Computing Lab (2024) — https://blog.vllm.ai/2024/06/15/vllm-one-year.html
- "The Cost of Inference: How the Market for LLM APIs is Evolving" — Latent Space (2025)
- "2025 LLM Inference Survey: Frameworks, Hardware, and the State of the Art" — Modal (2025) — https://modal.com/blog/llm-inference-survey-2025
- "Transformer Inference: From BERT to DeepSeek V3" — Chris Deotte, NVIDIA Developer Blog (2025)
- "LLM Inference in the Consumer Era" — LocalAI 博客 (2025) — https://localai.io/blog/consumer-inference-2025
- "The Future of LLM Inference: Reasoning-Time Compute Changes Everything" — Scale AI Blog (2026) — https://scale.com/blog/llm-inference-reasoning-time

---

> **方法论说明**：本报告使用横纵分析法（Horizontal-Vertical Analysis），由数字生命卡兹克提出。纵向追时间深度（从 2018 年 FasterTransformer 到 2026 年推理即服务的完整发展历程），横向追同期广度（四大推理引擎、量化方案、硬件生态的全面对比），最终在交叉处产出新的判断：推理正在从"成本中心的支撑工程"转变为"能力核心的计算基础设施"。
>
> 写作沿用了 Transformer 主报告和 MoE 报告的叙事风格——以具体事件和决策为锚点，在技术细节与行业叙事之间寻找平衡，避免咨询报告式的空洞概括。